

Classification of OS

Interaction Mode:

Batch processing:

- The OS runs a set of tasks all at once, without interacting with the user during execution.
- You submit jobs and the OS executes them in sequence.
- The user waits for the output; there's no "live feedback."

How it works:

- Jobs are collected in a queue.
- The OS schedules them one after another (or sometimes in groups).
- Each job gets exclusive CPU attention while running.
- Once finished, the results are returned (printed, saved, or displayed later).
- Each task gets 100% of the CPU until the task is finished

Examples:

- Banks collect all transactions for a day/week/month and run a batch job overnight to create account statements.
- Computes salaries for all employees at once. HR staff submits the data at the end of the month, the system processes it, and generates paychecks and reports later.

Time sharing:

The OS allows multiple users or tasks to interact with the computer at the same time.

The OS rapidly switches between tasks, giving the illusion that everyone has the computer all to themselves.

How it works:

- The CPU time is divided into small slices for each user/task.
- The OS switches very quickly between tasks (milliseconds).
- Users feel as if their program is running continuously.
- Multiple users can work concurrently on the same system.

Examples:

- You type, edit, and see changes instantly. OS switches between background tasks (like spell-check, auto-save) without slowing you down.
- Browsers like Chrome run multiple tabs simultaneously. You can interact with one tab while the OS runs background downloads or scripts on other tabs.

Terminologies:

Process or task are synonyms: it means a program in execution.

Program is a set of codes or instructions waiting to be run.

A Job: typically used in batch systems and Includes one or more programs

Single Tasking Operating System:

Definition

- A single-tasking OS can run only one program at a time on a CPU core.
- The CPU is entirely dedicated to one task until it finishes or is manually interrupted. Even if you have 3 cores and only one task the OS will only use 1 core to execute one program at a time.

Technical Detail

- CPU Execution: One program occupies the CPU; no other processes can run.
- No multitasking: Other programs must wait until the current program finishes.
- Memory Use: Simple, because only one program's data and instructions need to be in memory.

Real-World Example

- If you run a game on MS-DOS, you cannot simultaneously run a word processor or web browser (there weren't web browsers yet!).
- To run another program, you had to exit the current one.

Multi-tasking Operating System:

Definition

- A multitasking OS allows a CPU to manage multiple processes in a way that makes it look like they are running simultaneously.
- Technically, a single CPU core still executes only one process at any instant, but rapid context switching gives the illusion of parallelism.
- As an example if we have a processor (CPU) with 4 cores and 4 tasks to be ran each core will execute a task, if we have 5 then slicing will happen and cores will switch between tasks.

Technical Detail

- CPU Execution: Only one process runs at a time per core, but the OS switches tasks rapidly (milliseconds).
- Scheduling: OS uses algorithms like Round-Robin, Priority Scheduling, or Multilevel Queues to decide which process runs next.
- Memory Management: OS keeps track of multiple processes, each with its own memory space and resources.

Real-World Examples

- Windows, macOS, Linux:
 - You can browse the web, play music, and edit documents simultaneously.
- Server Context:
 - Linux servers run databases, web servers, and background scripts at the same time.

Key Concept: Pseudo-Parallelism

- True parallelism only happens if a CPU has multiple cores (or multiple CPUs).
- On a single-core CPU, multitasking is pseudo-parallel, achieved via time slicing and rapid context switching.
- Users perceive multiple programs running simultaneously, even though the CPU is only executing one at any instant.

Single User OS:

- Definition: Only one user can actively use the system at a time.
- Tasks: Can still run multiple programs (multitasking), but all belong to the same user.
- Examples: MS-DOS, Windows 95/98, early OS/2.
- Use Case: Personal computers and laptops.

Multi-User OS:

- Definition: Multiple users can use the system simultaneously, each with their own session.
- Tasks: OS manages CPU, memory, and resources for all users concurrently.
- Examples: Linux/Unix servers, Windows Server, mainframes.
- Use Case: Servers, enterprise systems, shared computing environments.

Memory Address:

What is a 32 bits OS: well the OS controls and manages the RAM, the RAM is already split into small boxes that each can store 1 byte. The OS of 32 bits can address and manage 2^{32} of those boxes which is $2^{32} * 8 \text{ bits} = 4 \text{ GB}$ this mean that the rest of those 4 GB assuming you have 8 GB RAM remains invisible

To clarify more, if you have the same 8 GB RAM and then you have 64 bits OS, your OS can address 2^{64} boxes (addresses) of your RAM meaning $2^{64} * 8 \text{ bits} = 8 \text{ GB}$.

Computer System with User Interaction:

This consists of a human giving an input through an input device,

- Input: any data or commands passed by the user, in real life input must be processed immediately
- The computer system takes input process it and produces output
- Hardware is The physical components of a computer that execute instructions and manage input/output.
 - CPU (Central Processing Unit) – executes program instructions
 - RAM/Cache – temporary storage for running programs
 - Storage devices (HDD/SSD) – long-term storage
 - I/O devices – keyboard, mouse, monitor
- Software: The programs that tell the hardware how to perform tasks.
 - **Operating System (OS):**
 - Manages hardware resources
 - Provides scheduling for tasks
 - In real-time systems, a Real-Time OS (RTOS) ensures tasks execute within deadlines
 - **Application Software:**
 - Performs user-specific or domain-specific tasks
 - Interacts with OS and hardware to produce results
 - Just your well known apps like figma, word
- Output: Output is the information, result, or action generated by the system.

Computer System For Control (embedded systems)

Embedded System – Refined Understanding

An embedded system is:

- A dedicated computer system built into a machine or device.
- Its purpose is to monitor, process, and control some part of the environment or the device itself.

How It Works

1. Sensors (Input):

- Detect physical variables (temperature, speed, distance, etc.)
- Convert these variables into electrical signals that the embedded system can read.

2. Embedded System (Processing / Brain):

- Hardware: Microcontroller, memory, I/O ports, timers, etc.
- Software: Control algorithms and sometimes a real-time operating system (RTOS)
- Reads the sensor signals
- Processes them using control logic or algorithms
- Decides what action or reaction is needed

3. Actuators (Output):

- Receive commands from the embedded system
- Perform physical actions that affect the environment or the device (motors, valves, LEDs, heaters, etc.)

4. Closed-Loop Control:

- After actuation, sensors measure the environment again → embedded system adjusts its actions continuously.

Slide Component	Description	Embedded System Correspondence
Environment – Technical Process (TP)	The physical system or process being monitored or controlled	The “environment” the sensors observe
Sensors	Measure physical variables and convert them into electrical signals	“Eyes” of the system; provide request r_i to the embedded system (not part of the ES)
Embedded System	Dedicated system that processes sensor data and decides actions	The “brain”; includes hardware + software
Software	Programs running on the embedded system implementing control logic	Part of the embedded system; computes the reaction
Hardware	Microcontroller, memory, and I/O interfaces	Physical part of the embedded system executing software
Actuators	Devices that act on the environment (motors, valves, heaters)	“Hands” of the system; carry out the reaction
Request r_i → Reaction	Flow of information from sensor to actuator	Sensor sends request → embedded system decides → actuator acts → environment changes

$$\text{Reaction Time}_i = t_{\text{sense},i} + t_{\text{E},i} + t_{\text{actuate},i}$$

In my words: reaction time of an embedded system is equal to time for sensor to read the environment the time of the system to process the data and decide, the time for the actuators to act.

Reaction Time in an Embedded System

Definition:

The reaction time is the total time it takes for the system to respond to a change in the environment. It consists of three main components:

1. Sensor Time (t_{sense}):
 - Time for the sensor to detect a change in the environment and send a signal to the system.
 - Example: A temperature sensor taking a few milliseconds to detect a rise in temperature.
2. Execution Time (t_{E}):
 - Time for the embedded system to process the sensor data and decide on an action.
 - Example: Microcontroller running a control algorithm to decide how to adjust a motor.
3. Actuation Time (t_{actuate}):
 - Time for the actuator to respond to the command from the system and physically affect the environment.
 - Example: Motor starting to move, valve opening, or heater turning on.

Soft Real Time and Hard Real Time Systems

Soft Real Time Systems:

- Soft real-time systems should meet their deadlines, but missing them occasionally is acceptable. If a task is late, the system still continues to function—just with reduced quality.
- Missing some deadlines is tolerated and occasionally happens
- Missing deadlines lead to: lower quality, lower customer satisfaction, very slight financial loss.
- Examples of such systems would be: media player, telephone systems, vending machine.

Hard Real Time Systems:

- hard real time systems on the other hand is time strict and should complete its task within a set of time boundary where execution time is less than the maximum time set. Missing the deadline once is not accepted.
- Missing the deadline for the execution might cause hardware damage, human life threats, and huge financial risks.
- Examples for such systems: aircraft flights, nuclear plants systems, medical systems, monitoring systems, trading systems.

Thin kernels:

- **Architecture:**
 - Hardware then thin kernel then operating system and the rest of the kernel followed by the user space for real time then the user space for not real time applications.
- **Functions:**
 - The thin kernel is the first line that intercepts hardware input or signal, the thin kernel decide whether the task is time critical or not then execute it immediately if its, and if its not then it passes it to the normal kernel.

Nano Kernels:

- **Architecture:**
 - Hardware → Nano kernel → Operating system services (mostly in user space) → User space for real-time tasks → User space for non-real-time tasks
- **Functions:**
 - The nano kernel is the first line that handles hardware interrupts and signals.
 - It decides whether a task is hard real-time.
 - If it is, the nano kernel executes it immediately or schedules it with strict timing guarantees.
 - If it is not, the nano kernel forwards the task to user-space OS services for normal handling.
 - Almost all OS services, including drivers, file systems, and networking, run in user space, keeping the kernel extremely minimal.

Kernel Vs OS

Kernel:

- Core part of the OS
- Manages CPU scheduling, memory management, and hardware interrupts
- Provides inter-process communication (IPC)
- Handles basic device access and protection mechanisms
- Ensures real-time or priority-based execution (if supported)

OS (Operating System):

- Kernel (core functionality) plus:
 - Device drivers (for hardware like printers, graphics, network cards)
 - System libraries (standard functions for applications)
 - User interfaces (CLI, GUI)
 - System utilities (file managers, backup tools, network tools)
 - APIs (for applications to safely interact with kernel services)

Main Features of the OS:

- Users and Groups, security and authorization, User Interface- Data - Memory- Processes/Tasks, synchronisation and cooperation (include communication)- Hardware, all other than CPU and memory.

User Mode Vs Kernel Mode:

User Mode:

Other names: Unprivileged mode, non-privileged mode

Purpose: Safe environment for normal applications that prevents them from directly accessing hardware or critical memory.

Key Characteristics:

- User programs run here: browsers, text editors, media players
- Interfacing to OS: Programs call the OS via system calls or APIs to perform tasks like reading a file or sending data over the network
- Isolated virtual memory space: Each program has its own memory, so one app cannot crash another
- Rights management: The OS controls what the program can do

Kernel Mode:

Other names: Privileged mode, supervisor mode

Purpose: Full control mode for the OS to manage hardware and core system functions.

Key Characteristics:

- Full access to hardware: CPU, memory, storage devices, network interfaces
- Access to all physical memory
- Can execute critical OS instructions that user mode cannot
- Handles interrupts, scheduling, and memory protection

Monolithic OS:

An OS architecture in which the kernel handles most of the operating system services, including device drivers, file systems, and networking, while the user application layer handles only standard applications.

Advantages: faster and single source

Disadvantages: a simple error may crash the entire kernel and so the system, security risks and hardware damage

Microkernel OS:

An OS architecture in which the kernel is minimal, handling only essential functions such as CPU scheduling and memory management, while most operating system services, like drivers and file systems, run in user space.

Advantages: enhanced security, if one service crash that does not crash the entire system

Disadvantages: lower efficiency for the fact that each application requires IPC, Microkernels are modular and safer, but the separation between kernel and drivers makes fixing or updating their communication more complex.